Principles of Robot Autonomy I

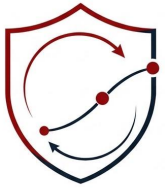
CS 237A / AA 274A / EE 260A / ME 274A

Course overview, Logistics,

Environment, Robot, and Agent Representations



Stanford University



SIA Lab

Safe and Intelligent Autonomy Lab





IPRL



Team

Instructor:



Prof. Somil Bansal

Course Assistants:



Carlota Pares Morlans



Joseph Lee



Aditya Kothari



Kabir Cheema

Email questions to: `cs237a-aut2627-staff@lists.stanford.edu`

Robot Autonomy

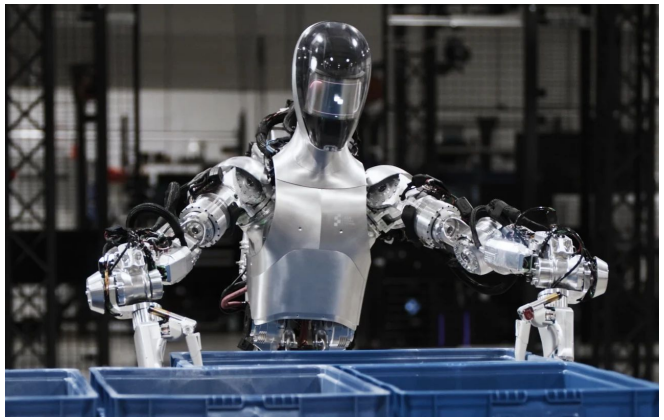
Waymo Self-Driving Car



Astrobee - NASA



Figure AI humanoid





Mobile Aloha





Course goals

To learn the *theoretical*, *algorithmic*, and *implementation* aspects of main techniques for robot autonomy:

1. Gain a fundamental knowledge of the “autonomy stack”
2. Be able to apply such knowledge in applications and research using ROS2
3. Devise novel methods and algorithms for robot autonomy

Main focus: *robot navigation,* not *robot manipulation*

Other robotics resources at Stanford

Manipulation Courses:

CS223A: Intro to robotics (Khatib)

CS225A: Experimental Robotics (Khatib)

CS327A: Advanced Manipulation (Khatib)

CS237B: Robot Autonomy II (Bohg, Sadigh, Pavone)

CS326: Advanced robotic manipulation (Bohg)

CS227: Robot perception (Song)

ME314: Robotic Dexterity (Kennedy)

Navigation Courses (including Localization, Mapping, SLAM):

CS237B: Robot Autonomy II (Bohg, Sadigh, Pavone)

ENGR205: Introduction to Control Design Techniques (Bansal)

AA203: Optimal and Learning-based Control (Pavone)

AA212: Advanced Feedback Control (Schwager)

AA275: Navigation for Autonomous Systems (Gao)

AA273: Filtering and Estimation for Robot Perception (Schwager)

CS 224R: Deep RL (Finn)

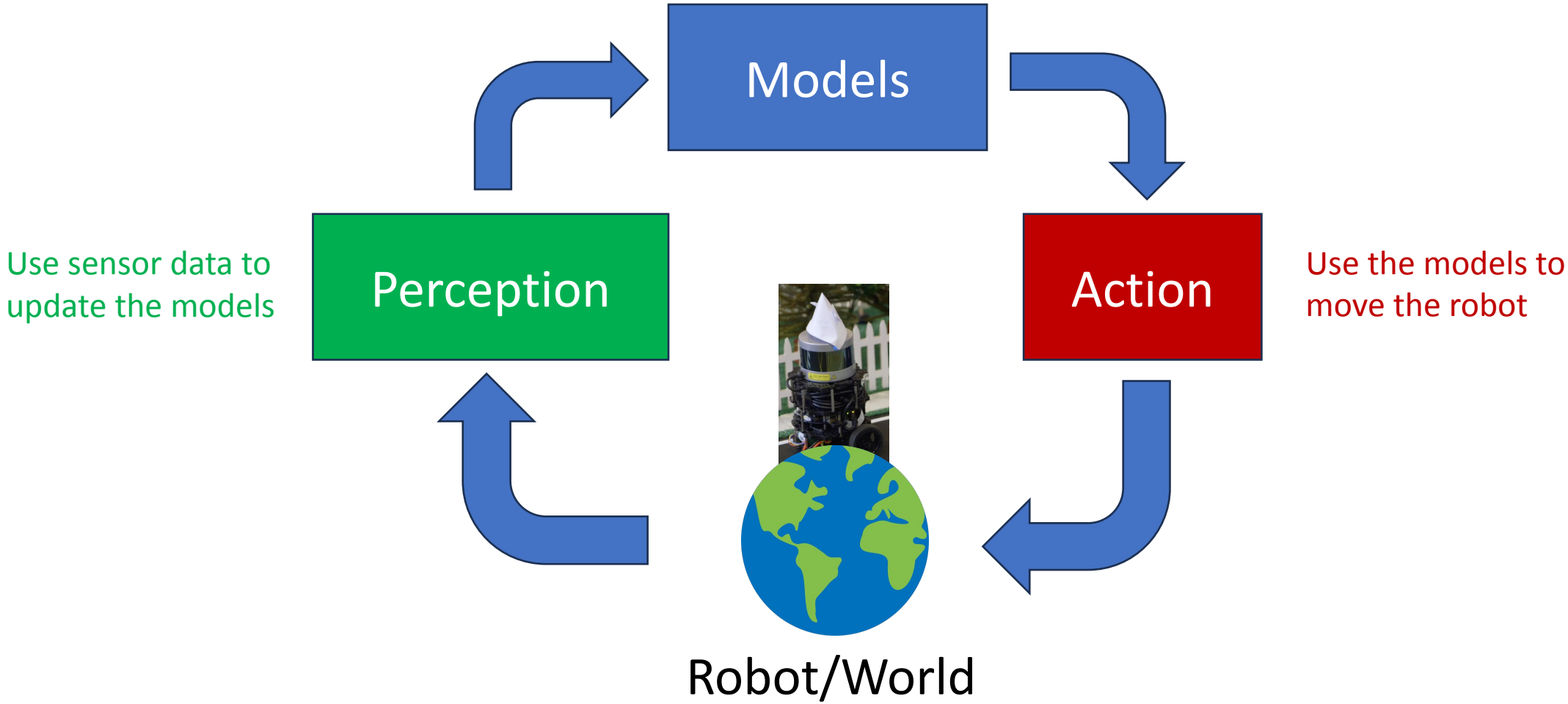
ME 326: Collab Robotics (Kennedy)

CS 345: Building AI-Enabled Robots (Liu)

AA 276: Principles of Safety-Critical Autonomy (Bansal)

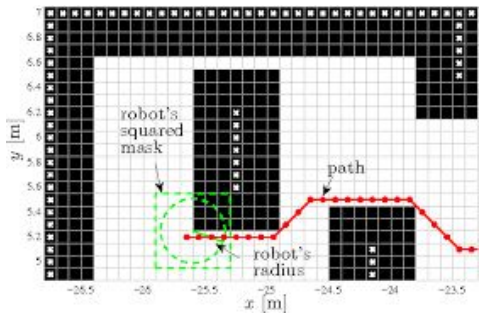
Stanford Robotics Center (SRC): https://src.stanford.edu/

Full Stack Autonomy: the Perception-Action Loop

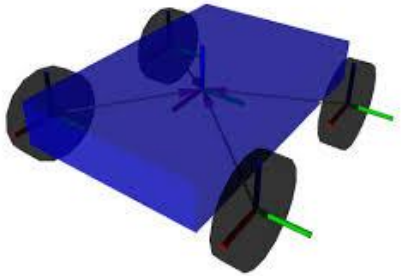


Models: Environment, Robot, Agent Representations

Maps:



Robot models or simulators:

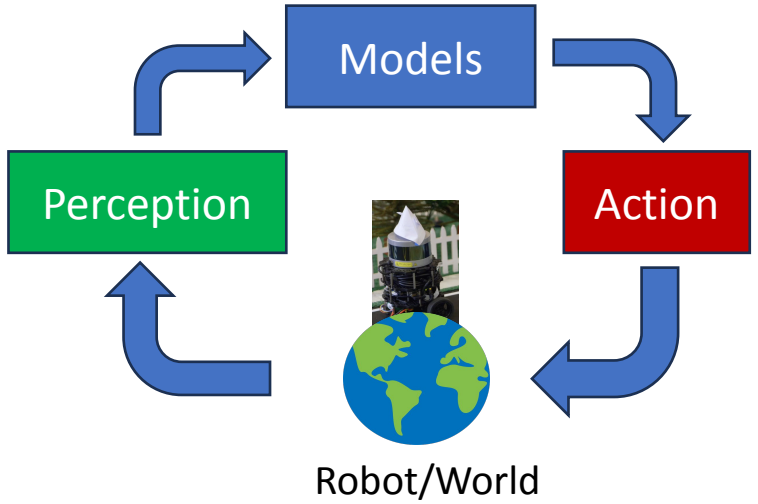


Agent models or simulators:



Maps:

- Location and geometry of obstacles
- Free vs occupied space
- Global coordinate frame
- Uncertainty quantification



Robot models:

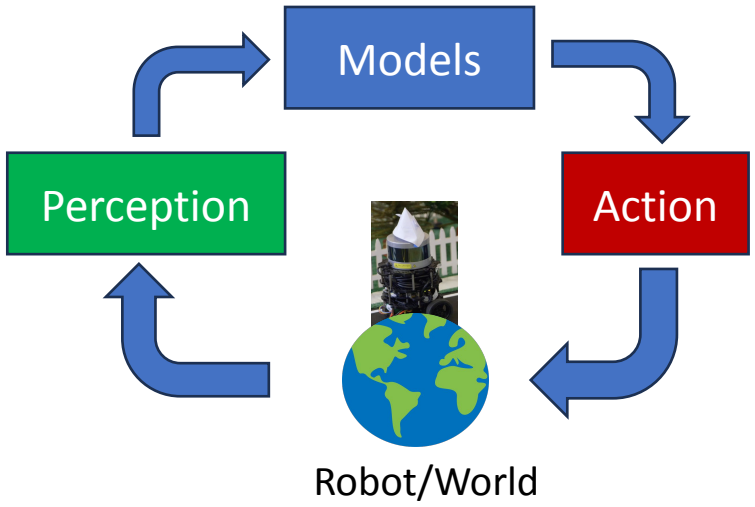
- Kinematics/dynamics
- Geometry
- Rigid body physics simulators

Object/Agent Models:

- Motion prediction models (how will it move)?
- Geometry models
- Uncertainty quantification

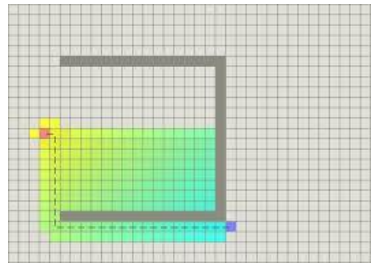
Action: Planning and Control Stack

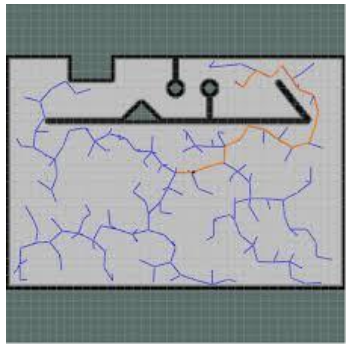
Use the models to move the robot



Path planning:

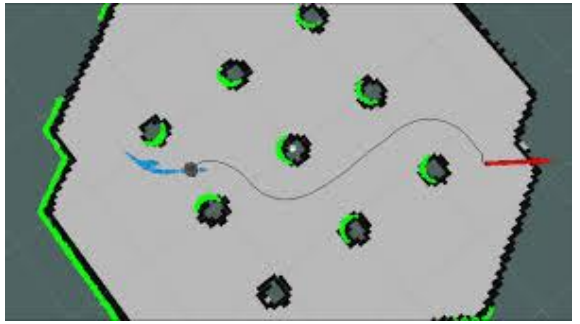
- A* planning on a graph
- RRT, RRT*





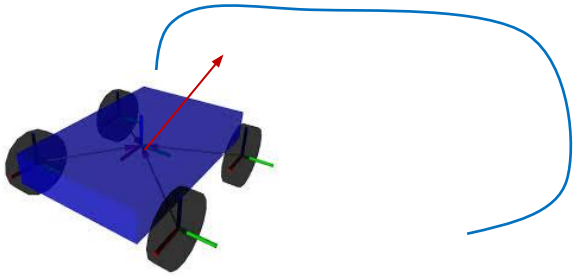
Trajectory optimization:

- Optimal control
- Numerical solvers
- **AA203: Opt and Learning Cont (Pavone)**



Trajectory following:

- Kinematic/dynamic models
- PID control
- LQR, gain scheduled LQR
- **ENGR205: Int. to Cont Design (Bansal)**
- **AA212: Adv. Feedback Cont (Schwager)**

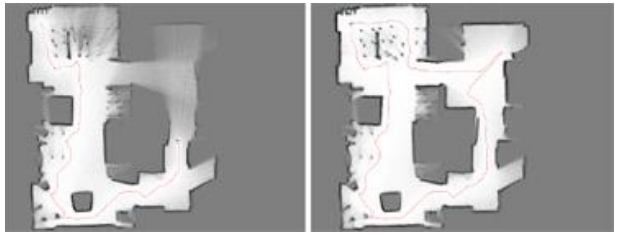


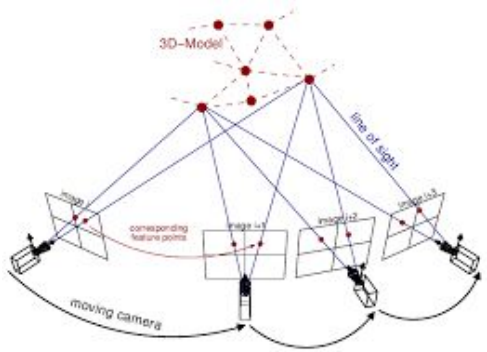
Perception Stack: Computer vision, filtering, SLAM

Use sensor data to update the models

Localization, Mapping, Tracking:

- EKF/Monte Carlo localization
- Occupancy grid mapping
- Pose graph optimization
- Tracking (EKF and Particle Filter)
- **AA273: Filtering (Schwager)**
- **AA275: Navigation (Gao)**





Information extraction

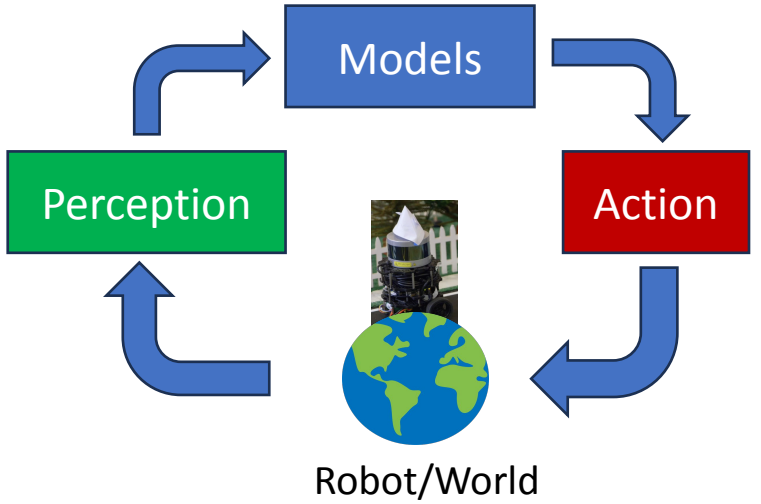
- Computer vision: features, correspondences, Structure from Motion (SfM), depth
- Lidar scan matching, ICP
- **CS231A: Comp Vision**



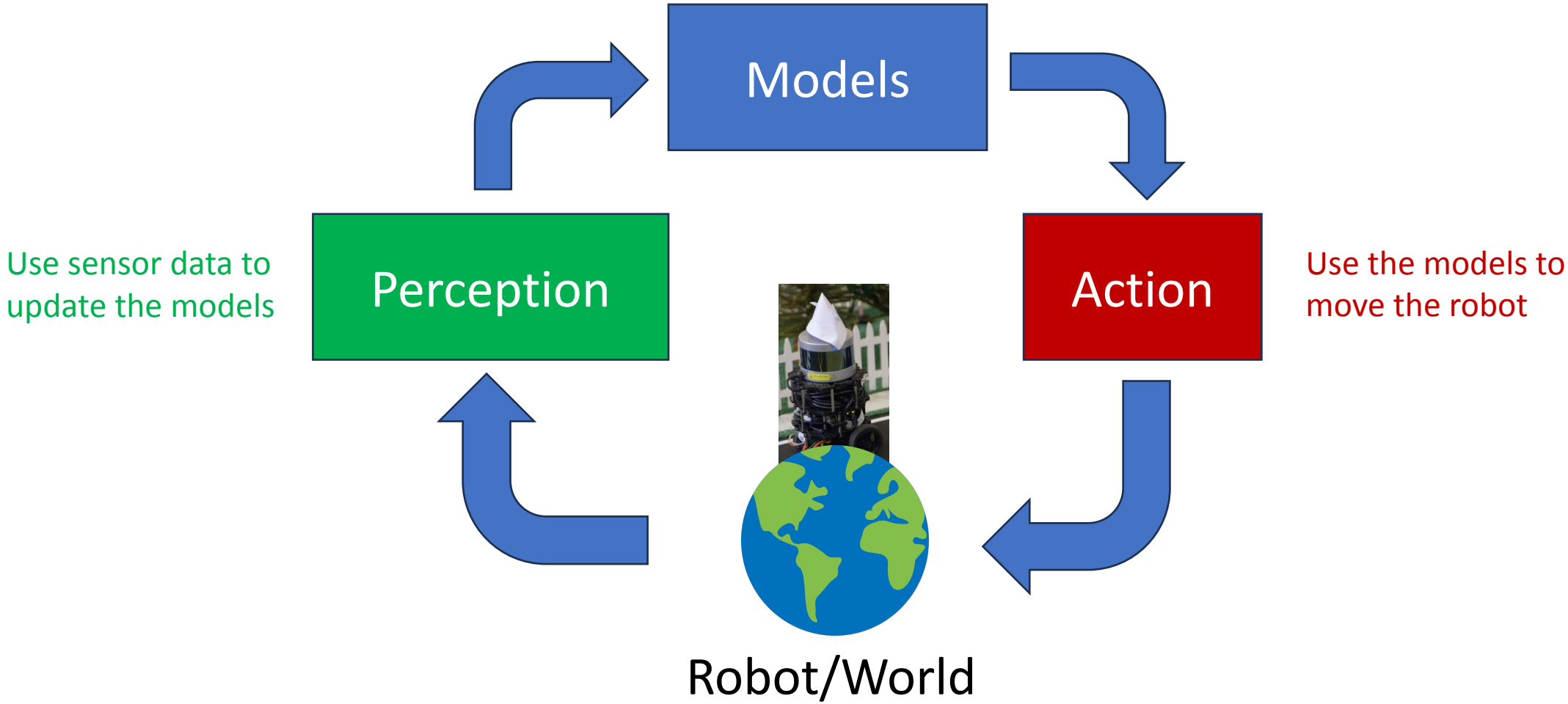
Sensors:

- RGB Cameras, RGB-D/stereo cameras, Lidar
- IMU, GPS, wheel encoders





Full Stack Autonomy: the Perception-Action Loop



But don’t robots use ML/AI?!

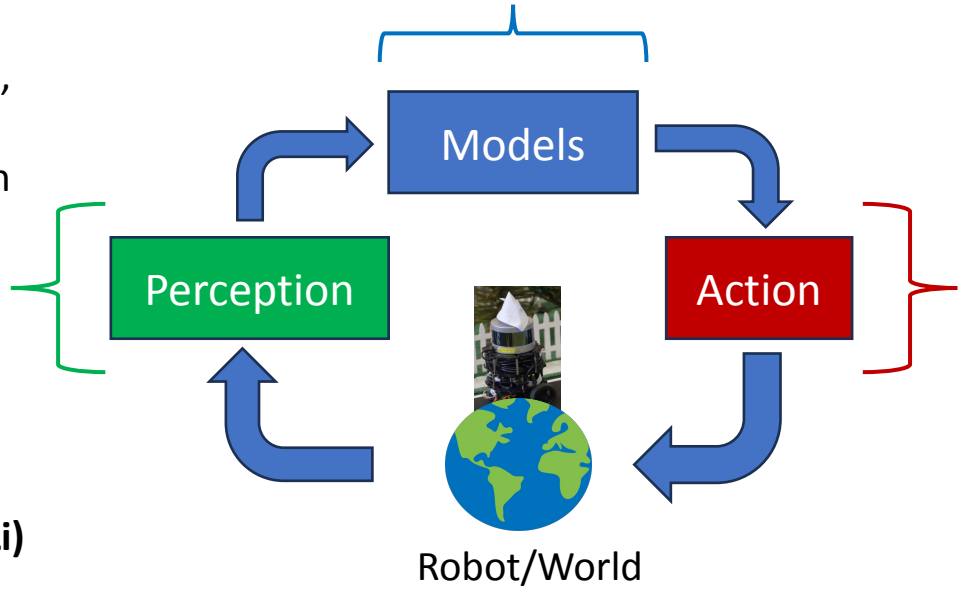
Yes! But only a little in this class. We treat classical, model-based autonomy.

Learning Based Perception Models:

- Extremely common, widely used in real robots, especially in semantic tasks
- Object detectors (YOLO, OWL-ViT)
- Object segmentation (SAM, SAM2)
- General vision-semantic features (DINO, CLIP)
- Geometric tasks SfM, SLAM, localization (VGGT, DUSt3R, SpatialTracker, Foundation Pose), we still often use classical techniques
- **CS227: Robot Perception (Song)**
- **CS231A: 3D Computer Vision (Bohg, Savarese)**
- **CS231N: Deep Learning for CV (Adeli, Li)**

Learning for Models

- Learned Maps: Neural Radiance Fields (NeRFs), Gaussian Splatting (GSplat)
- “World Models”: Learn function from action-image history to next image. Often learns from an existing simulator (not real world).
- Model Based Reinforcement Learning (MBRL)
- **AA273: State Estimation and Filtering (Schwager)**
- **AA 275: Navigation for Autonomous Systems (Gao)**

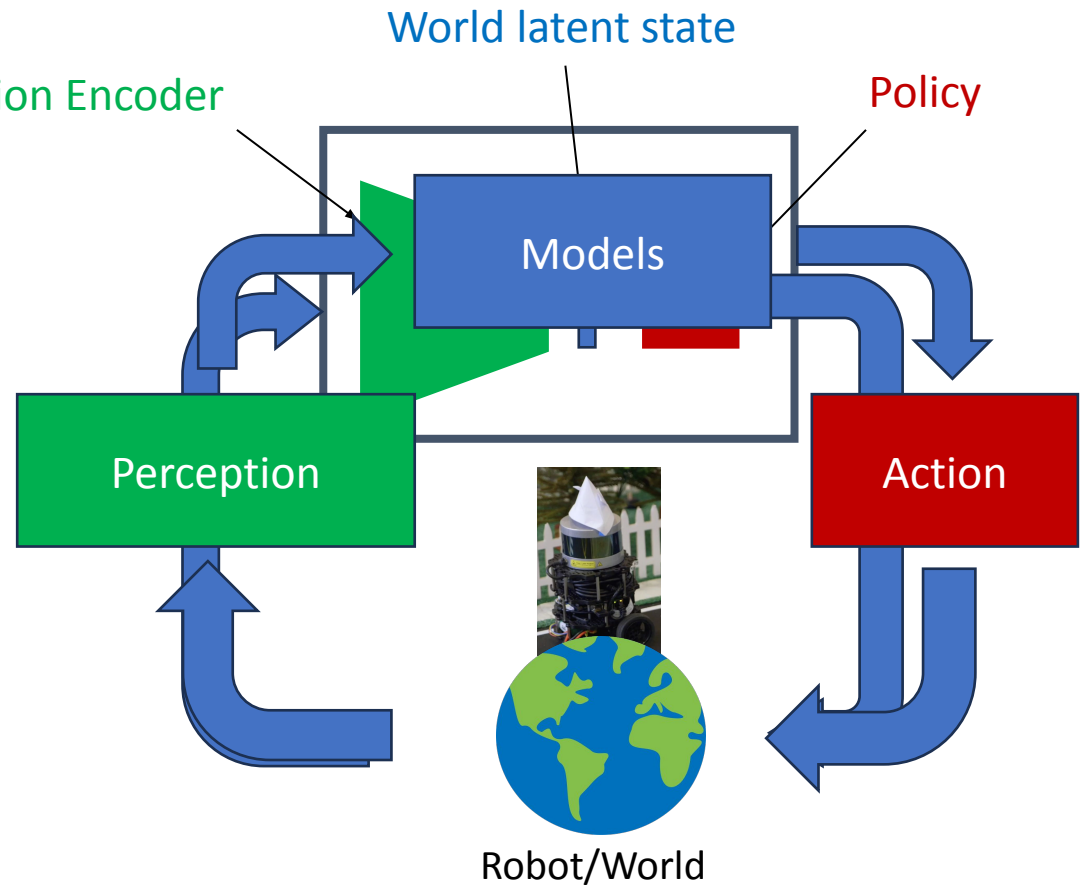


Reinforcement Learning (RL):

- Learn policy from state to action through incremental improvement
- Often heavily dependent on a simulator
- Sim-to-real a major challenge
- Often still relies on classical lower-level control layers
- **CS 224R Deep RL (Finn)**
- **CS 234 RL (Brunskill)**
- **AA203: Opt and Learning Cont (Pavone)**

What about End-to-End Learning?

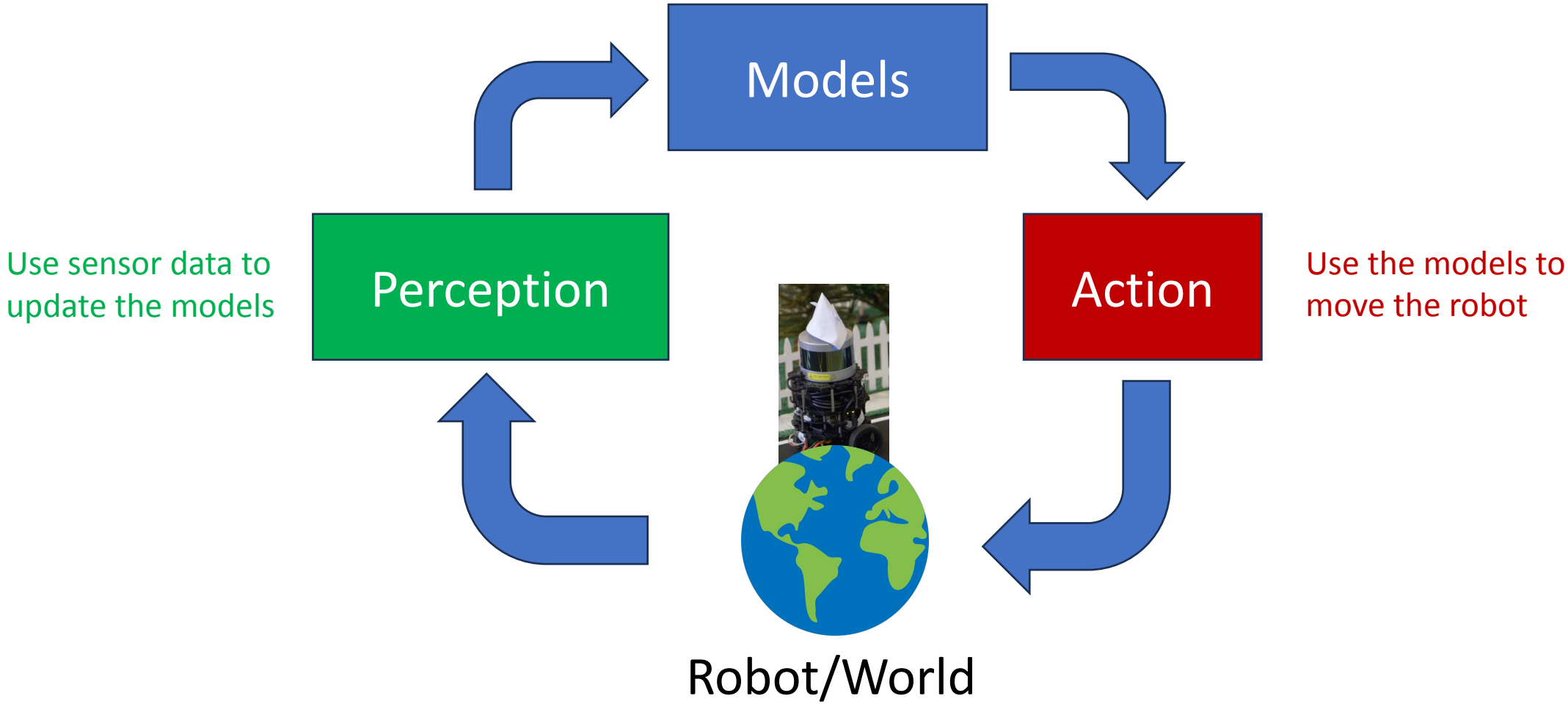
Yes, very cool! But not in this class.



End-to-End Visuo-Motor Policy

- Still have the three basic components internally in learning architecture
- Imitation learning (IL)/Behavior Cloning (BC)
  - Learn visuomotor policy from human demonstrations
  - Very successful in recent research!
  - **CS 237B Robot Autonomy II (Bohg, Sadigh, Pavone)**
- Reinforcement learning (RL)
  - Usually learned in simulators
  - Quite data-hungry
  - Sim-to-real is often a major challenge
  - **CS 224R: Deep RL (Finn)**
  - **CS 234: RL (Brunskill)**
  - **CS123: AI Enabled Robots (Liu)**

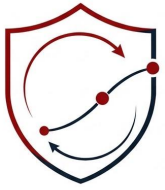
Full Stack Autonomy: the Perception-Action Loop



Logistics



Stanford University



SIA Lab
Safe and Intelligent Autonomy Lab





IPRL



Autonomous Systems Lab

Course structure

- Four modules
  1. Models: maps, kinematic and dynamic robot models
  2. Action: planning and control
  3. Perception: robot vision, point clouds, Kalman filtering, SLAM
  4. Advanced topics: NeRFs/3DGS, Imitation Learning/RL, VLAs
- Extensive use of the Robot Operating System (ROS2)
- Requirements
  - CS 106A or equivalent
  - CME 100 or equivalent (for calculus, linear algebra)
  - CME 106 or equivalent (for probability theory)
  - A pre-knowledge self-assessment will be sent out on Canvas today

Schedule

Models

Perception

Date	Topic	Homework	Lab
09/22	Course overview, perception-action loop, maps		Lab 0: Install ROS
09/24	Maps, Robot geometry, Coordinate frames and SE(2)/SE(3) transforms	HW1 out	
09/29	Collision, C-space, motion models. Path planning I: A*		Lab 1: Command line, Git, Python
10/01	Path planning II: RRT, RRT*		
10/06	Trajectory optimization		Lab 2: ROS basics
10/08	Trajectory following: PID, LQR, gain scheduled LQR	HW1 due, HW2 out	
10/13	Robotic sensors: IMU, lidar, cameras, RGB-D. Point clouds & ICP		Lab 3: RViz, Turtlebot
10/15	Pinhole camera models, camera calibration	HW2 due, HW3 Out	
10/20	Structure from Motion (SfM), features, RANSAC		Lab 4: Heading controller
10/22	Learning based perception, semantic perception		
10/27	SLAM intro, factor graphs, PGO	HW3 due	Lab 5: Nav to goal
10/29	Midterm (in-class)		
11/03	*No Lecture (Democracy Day)*	HW4 out	
11/05	Pose graph opt, bundle adjustment		
11/10	Bayes Rule, RVs, Occ. mapping		Lab 6: Object detection
11/12	Occ. mapping, frontier exploration	HW4 due, HW5 out	
11/17	Bayesian filter, Gaussian RVs, Kalman Filtering		Lab 7: Frontier exploration
11/19	EKF, EKF localization, mapping, SLAM, UKF		
11/24	*No lecture (Thanksgiving)*		
11/26	*No lecture (Thanksgiving)*		
12/01	Particle Filtering, Monte Carlo localization	HW5 due	Lab 8: Makeup
12/03	Advanced Topics: Imitation learning, VLAs, 3DGS for sim2real, world models		

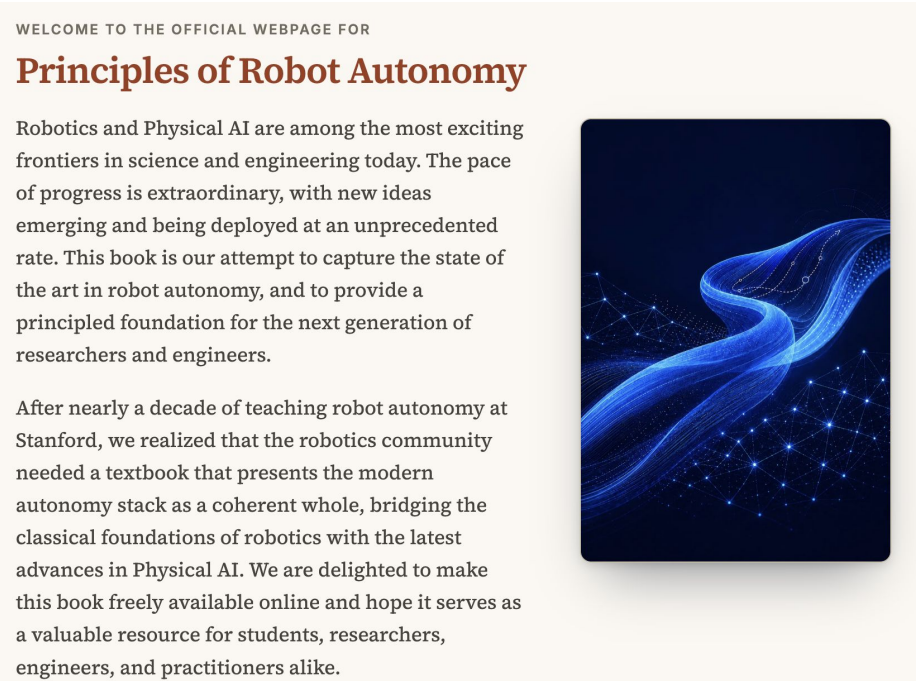
Action

Advanced topics

Logistics - Lectures

- Tuesdays and Thursdays, 1:30pm – 2:50 (Skilling Aud)
- Recordings will be made available to all students on Canvas.

- Main reference:
  - D. Gammelli, J. Lorenzetti, K. Luo, G. Zardini, M. Pavone. Principles of Robot Autonomy. 2026.
  - Available at https://arxiv.org/abs/2608.03496
  - Book website: https://porabook.com/
  - Each lecture will include references to the corresponding sections of the book.

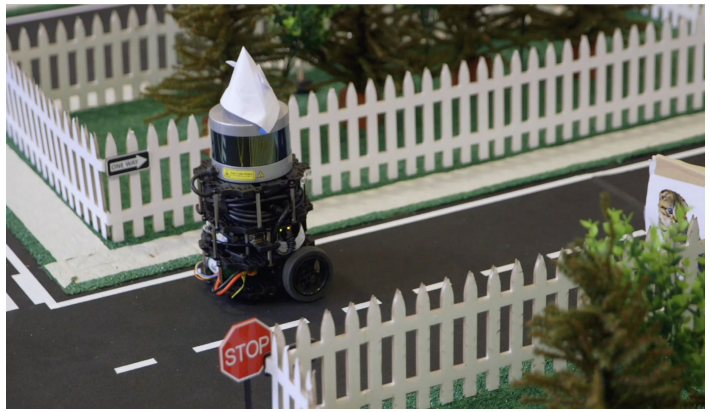


Logistics – Homework Assignments

- 5 assignments
- First homework out on Thursday
- ~2 weeks to submit on Gradescope
- Homeworks are crucial for sections, code development.
  - lecture introduces ideas, homework develops understanding and code, and laboratory work deploys those pieces on the robot.
- 4 late days throughout the quarter, no questions asked! No extensions beyond this, no exceptions (except OAE accommodations). So please plan ahead!
- Cooperation and discussion is encouraged, but solutions must be prepared individually. Add names of classmates who you collaborated with. Copying from other students or other sources is considered a case of academic dishonesty.
- Need to be typeset in Latex!

Logistics – Sections

- 2-hour, once-a-week sessions starting Week 2
- Hands-on exercises that complement the lecture material, build familiarity with ROS,
- Progressively implement whole autonomy stack on turtlebot hardware
- Section sign-up sheet by webform is out now! (see Canvas announcement)
- Section timing details on Syllabus (see Canvas and course website)





Logistics – Exams

- Midterm exam: in-class during Week 6.
- Final exam: during the finals week.

AI Policy

Absent a clear statement from a course instructor, use of or consultation with generative AI shall be treated analogously to assistance from another person. In particular, using generative AI tools to substantially complete an assignment or exam (e.g. by entering exam or assignment questions) is not permitted. Students should acknowledge the use of generative AI (other than incidental use) and default to disclosing such assistance when in doubt.

1. AI Coding Assistants: These are modern productivity tools integral to most software engineering projects. Use them but **understand your code! It is your responsibility to learn what the code means and how it works.**

2. Chat GPT, Gemini, Etc. on Homeworks or Exams (except coding): Don't do it. If you do, cite it and justify the reason. Points may be deducted. If you use it and do not cite it, we will escalate the issue.

Logistics - Grades

- 20% Homeworks (5 x 4% each)
- 35% Sections (7 x 5% each)
- 17% Midterm
- 28% Final

Logistics

- Office hours: See Syllabus on Canvas
- Course websites:
  - **Cavas:** For syllabus, announcements, lecture videos, slides, and supplementary material
  - **Edstem:** For course-related questions and discussion
  - **Gradescope:** For homework submissions and grading
  - **email:** cs237a-aut2627-staff@lists.stanford.edu to contact the CS237A staff
  - **Legacy Website:** http://asl.stanford.edu/aa274a/ will be updated soon. Same info as Syllabus
- Syllabus has all the info!
- Class capacity, enrollment cap, waitlist

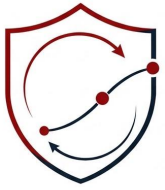
Announcements

- Sign up for section **by Thursday**. Ranked Choice – NOT first come-first-serve.
- Sign up if you are on waitlist and still want to take course.
- Homework 1 will be released Thursday (canvas)
  - due Thursday 10/08 (by gradescope)
- Section 0 will be released today. Watch Section 0 Notion Instructions- set up your Virtual Machine and install ROS 2.0.
- Pre-knowledge self-assessment out today (canvas)

Models:
Environment models, Robot models, Agent models



Stanford University



SIA Lab
Safe and Intelligent Autonomy Lab





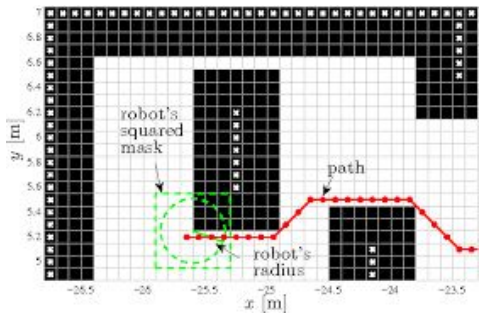
IPRL



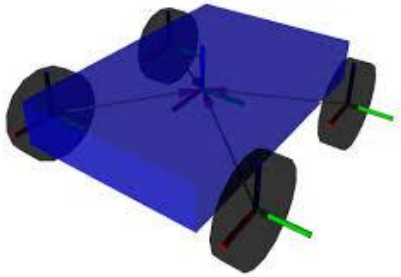
Autonomous Systems Lab

Models: Environment, Robot, Agent Representations

Maps:

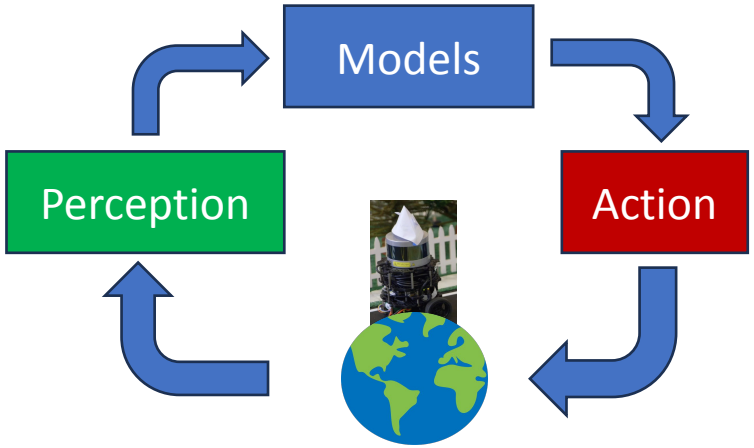


Robot models or simulators:



Agent models or simulators:

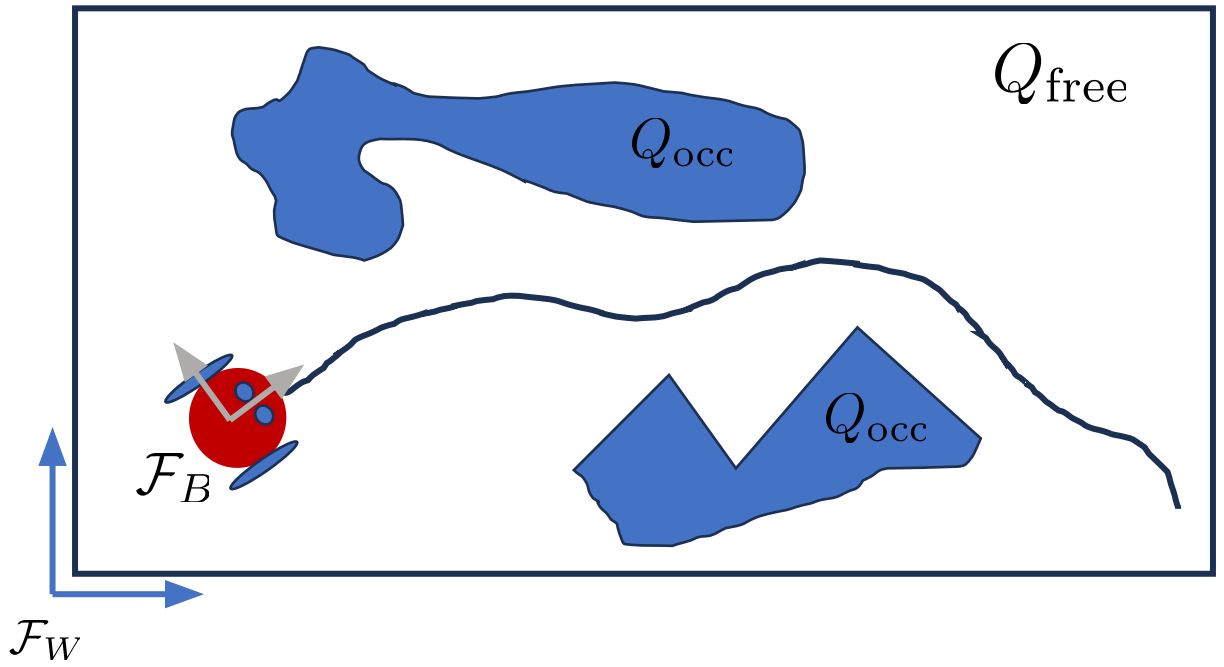




Maps

Purpose: To represent free and occupied space in a *fixed world frame*. Used for collision avoidance and path planning in robot navigation.

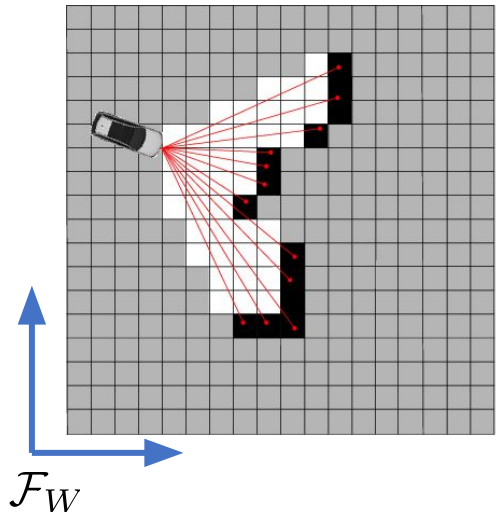
$$Q = Q_{\text{free}} \cup Q_{\text{occ}} \quad \text{so} \quad Q_{\text{free}} = Q_{\text{occ}}^{C}$$



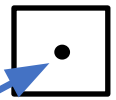
How to mathematically represent these sets?
How to update them from robot sensor measurements as the robot moves?

Occupancy Grid Map

2D:



Grid cell i:



Location $q_i = (q_i^x, q_i^y)$

State $x_i \in \{0, 1\}$

$$Q = Q_{\text{free}} \cup Q_{\text{occ}}$$

$$Q_{\text{free}} = \{q_i \mid x_i = 0\}$$

$$Q_{\text{occ}} = \{q_i \mid x_i = 1\}$$

Probabilistic occupancy grid:

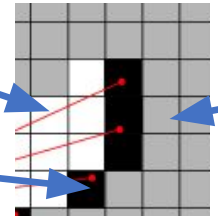
$$p_i = p(x_i = 1 \mid \text{sensor measurements})$$

$$p_i \in [0, 1]$$

Probability threshold:

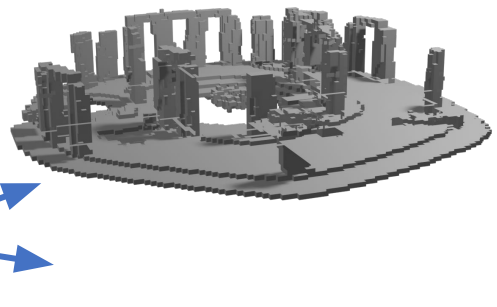
$p_i \leq 0.05$

$p_i \geq 0.95$



$0.05 \leq p_i \leq 0.95$

3D:



Log-Odds:

$$l_i = \log\left(\frac{p_i}{1 - p_i}\right) \in [-\infty, \infty]$$

$$l_i = 0 \iff p_i = 0.5$$

Makes updating with Bayes' rule very simple (we'll see later)

"Mapping": Update all p_i with Bayes' rule in real time using robot sensor measurements.

Point Cloud Mapping

2D:



Points represent occupied space:

$$q_i \in \mathbb{R}^2 \text{ or } \mathbb{R}^3$$

Native output of a lidar sensor.

$$Q_{\text{occ}} = \{q_i\}_{i=1}^{N}$$

Still need to transform from robot body to world frame.

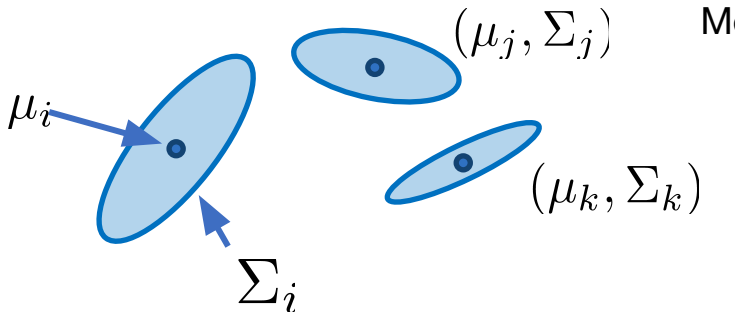
Probabilistic point cloud map:

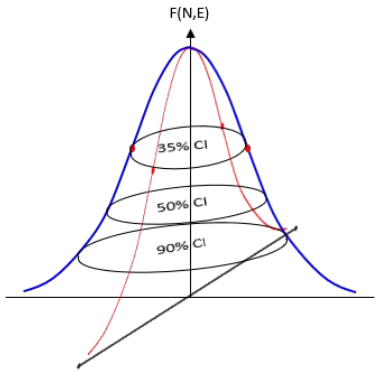
Multivariate Gaussian

$$p(q_i \mid \text{robot measurements}) = \mathcal{N}(\mu_i, \Sigma_i)$$

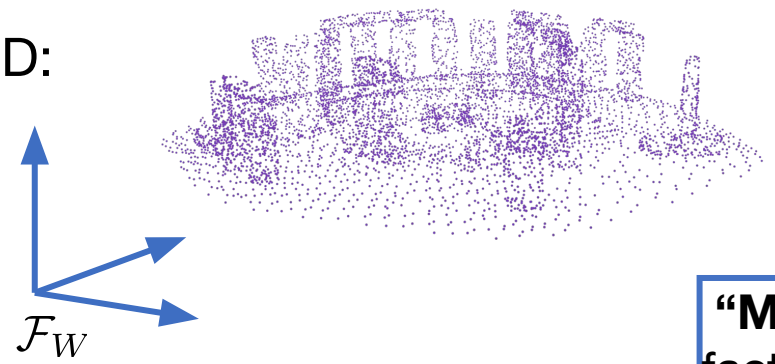
Mean vector

Covariance matrix





3D:



"Mapping": Update (μ_i, Σ_i) with a Kalman filter or a factor graph optimization solver.